COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	SECTION: CPE21S4	
DATE PERFORMED:	17/07/2026	DATE SUBMITTED:	17/07/2026
NAME:	Jullian Edrain R. Sevilla	INSTRUCTOR:	Engr. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Microsoft copilot, code visualizer

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: For hints and clarification

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: To give hints but not to fix the errors i have make

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Table 2-1

```
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Copy Assignment Operator
    Student& operator=(const Student &other) {
        std::cout << "Copy Assignment Operator Called." << std::endl;
        if (this != &other) {
            studentName = other.studentName;
            studentAge = other.studentAge;
        }
        return *this;
    }

    // Display Attributes
    void printDetails() {
        std::cout << studentName << " " << studentAge << std::endl;
    }
};
```

```
Copy Assignment Operator Called.
Destructor Called.
Zack 19
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\CPES4_Sevilla\.vscode>
```

Observation - The output shows the constructor creating student1, the copy constructor duplicating it for student2, and the default constructor for student3. When student3 = student2 runs, the copy assignment operator is called. Finally, destructors appear at the end, confirming proper cleanup. This demonstrates how the big three handle object creation, copying, and assignment in static memory.

Table 2-2

```
    // Array of Students
    const size_t j = 5;
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    Student studentList[j];
    for (size_t i = 0; i < j; i++) {
        studentList[i] = Student(namesList[i], ageList[i]);
        studentList[i].printDetails();
    }

    return 0;
}
```

```
Constructor Called.
Copy Constructor Called.
Constructor Called.
Copy Assignment Operator Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Carly 15
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Freddy 16
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Sam 18
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Zack 19
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Copy Assignment Operator Called.
Destructor Called.
Zack 19
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Cody 16
Destructor Called.
Destructor Called.
Copy Assignment Operator Called.
Destructor Called.
```

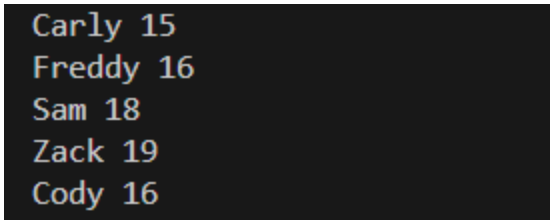
Observation - With the array version, constructors are first called for each element, then temporary objects are created and assigned inside the loop. The assignment operator copies values into the array, and each student's details print correctly. At program exit, destructors are called for all objects. This shows how arrays of objects are managed automatically, with constructors, assignment operators, and destructors working together.

Loop A

```
Copy Constructor Called.
Constructor Called.
Copy Assignment Operator Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
```

Observation - Loop A demonstrates dynamic allocation. Each call to new Student triggers a constructor, and the assignment operator copies the object into the static array. Adding delete ptr; ensures memory is freed properly.

Loop B



Observation - Loop B confirms that all student data was stored and accessed correctly. The printed output shows that the array elements hold the expected names and ages.

Modification - Added delete ptr; inside Loop A to prevent memory leaks.
Observation - The correction ensures all dynamically allocated memory is freed, making the program safe and efficient.